Simple Solutions – Recursion and Backtracking

Problem 1

a) Recursive Function

```
function findSize(folder):


    if folder is file:

        return file size


    total = 0


    for each item in folder:

        total = total + findSize(item)


    return total
```

Base Case

If item is a file, return its size.

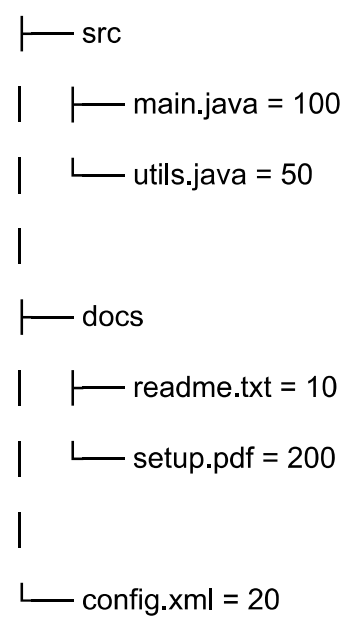
Recursive Case

If item is a folder, calculate size of all files inside it.

---

b) Execution

project



Calculation

src = 100 + 50 = 150

docs = 10 + 200 = 210

project = 150 + 210 + 20

Total = 380 KB

---

c) Time Complexity

O(N)

Because every file and folder is visited once.

---

d) Iterative Solution

Yes, it can also be solved using:

- Stack
- Queue

Difference

Recursion	Iteration
Easy to write	Better memory control
Uses call stack	Uses explicit stack
Can cause stack overflow	Safer for deep folders

---

e) Cycle Problem

If folder links back again:

A → B → C → A

program will run forever.

Solution

Use visited set.

if folder already visited:

    return

Problem 2

a) Word Search Backtracking

Steps

1. Start from matching letter
2. Move in all directions
3. Mark visited cell
4. If path fails, backtrack

Pseudocode

```
function search(row, col, index):


    if index == word length:

        return true


    if cell invalid:

        return false
```

mark visited

move in all directions

unmark visited

---

b) DREAM Path

Grid:

C A T S

O R E A

D E A M

E L L S

Path:

D → R → E → A → M

Coordinates:

(2,0) → (1,1) → (1,2) → (2,2) → (2,3)

---

c) Backtracking Process

- Choose correct letter
- Move to next letter

- If wrong path:
    - go back
    - try another direction

---

d) Time Complexity

O(N × M × 8^L)

Where:

- N = rows
- M = columns
- L = word length

---

e) Find All Paths

Instead of stopping after first answer:

store every valid path

Need:

- List to store answers

Problem 3

a) Search Space

Brute Force

8^8 = 16777216

One Queen Per Row

8! = 40320

Backtracking checks much fewer states because invalid positions are removed early.

---

b) Optimized N-Queens

Use 1D array.

board[row] = column

Example:

[1,3,0,2]

means:

Row 0 → Col 1

Row 1 → Col 3

Row 2 → Col 0

Row 3 → Col 2

Pseudocode

function solve(row):

```
    if row == n:
        print solution
        return

    for col in board:

        if safe:

            place queen

            solve(row + 1)

            remove queen
```

---

c) N = 4 Solution

board = [1,3,0,2]

Board:

. Q . .

. . . Q

Q . . .

. . Q .

---

d) Find Only One Solution

Stop recursion after first valid answer.

return true

This saves time.

---

e) Forbidden Squares

Before placing queen:

if blocked:

    continue

Complexity

Still:

O(N!)

But practical performance becomes better.

Short Summary

Recursion

- Function calls itself
- Needs base case
- Needs recursive case

Backtracking

- Try
- Explore
- Undo
- Try another path

Applications

- N Queens
- Sudoku
- Maze
- Word Search
- Graph Coloring

Backtracking is faster than brute force because it removes wrong paths early.